Top 50 Spring Boot Interview Questions

1. What is Spring Framework and why is it used?

Answer:

- Spring is a lightweight, open-source Java framework that provides a comprehensive programming and configuration model.
- It simplifies Java application development.
- It provides support for IoC, AOP, Transactions, JDBC, MVC, Security, Testing, etc.
- It is used to build enterprise applications in a modular and maintainable way.

2. What is Spring Boot and why was it introduced?

Answer:

- Spring Boot is an extension of Spring that simplifies the initial setup and development of Spring based applications.
- It was introduced to reduce the boilerplate code, complex configurations and setup time.
- It follows "Convention over Configuration" principle.
- It provides production-ready features out-of-the-box like embedded server, health checks, metrics, etc.

3. Spring vs Spring Boot – what is the difference?

Spring Framework	Spring Boot
• Requires manual configuration (XML / Java Config).	• Auto-configuration, minimal config required.
• No embedded server by default, need to deploy on external server.	• Embedded server (Tomcat/Jetty) by default.
• More boilerplate code.	• Less boilerplate code.
• Dependency management is manual.	• Uses "Starters" for dependency management.
• Setup is complex and time consuming.	• Quick setup and easy to start.

4. What are the main advantages of Spring Boot?

Answer:

- Quick and easy project setup.
- Auto-configuration reduces manual work.
- Embedded servers (Tomcat, Jetty, Undertow).
- Starters simplify dependency management.
- Production-ready features (Actuator, Health checks).
- Externalized configuration support.
- Easy testing and debugging.
- Follows modern best practices.

5. What is Spring Boot Starter?

Answer:

- Starters are a set of convenient dependency descriptors.
- They simplify build configuration by providing a curated list of dependencies.
- Example: spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-security.

Example:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

← Includes commonly needed dependencies automatically.

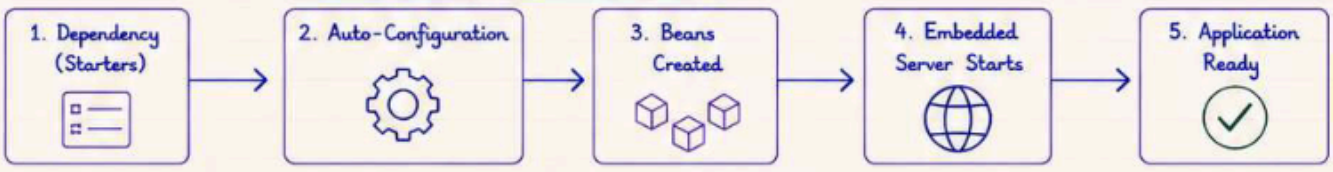
6. What is Spring Boot Auto-Configuration?

Answer:

- Auto-configuration is a feature that automatically configures the application based on dependencies in the classpath.
- It reduces manual configuration.
- It tries to guess and configure beans that are needed.
- You can override auto-configuration by providing your own configuration.

Note: Auto-configuration makes Spring Boot smart by configuring the application for you.

Spring Boot High Level Flow



Top 50 Spring Boot Interview Questions

7. What is IoC (Inversion of Control)?

Answer:

- IoC is a design principle where the control of object creation and dependency management is transferred from the application code to the Spring container.
- It helps in achieving loose coupling and easier testing.
- Spring IoC container is also known as Bean Factory.

Example:

Instead of creating objects like this,

```
UserService service = new UserService();
UserRepository repo = new UserRepository();
```

We let Spring container create and inject them.

Key Point: IoC = You don't control the objects, the container does.

8. What is Dependency Injection?

Answer:

- Dependency Injection (DI) is a way of providing dependencies (objects) to a class instead of the class creating them itself.
- It is a core concept of IoC.
- It promotes loose coupling and makes the application more testable.

Types of DI:

- Constructor Injection
- Setter Injection
- Field Injection

Example (Constructor Injection)

```
@Service
public class UserService {
  private final UserRepository repo;

  public UserService(UserRepository repo) {
    this.repo = repo;
  }
}
```

Key Point: DI = Dependencies are provided, not created.

9. Constructor Injection vs Setter Injection vs Field Injection?

Type	Details
Constructor Injection	• Dependencies are provided through constructor. • Ensures immutability. • Preferred and recommended.
Setter Injection	• Dependencies are provided through setter methods. • Allows re-injection. • Can lead to mutable objects.
Field Injection	• Dependencies are injected directly into fields using @Autowired. • Not recommended (harder to test).

Note: Constructor Injection is the best practice.

10. What is a Spring Bean?

Answer:

- A Spring Bean is an object that is instantiated, assembled, and managed by the Spring IoC container.
- Each bean has a unique id.
- The container takes care of its lifecycle, dependencies, configuration, etc.
- Beans are the building blocks of a Spring application.

Example:

```
@Service
public class UserService {
  // This class is a Spring Bean
}
```

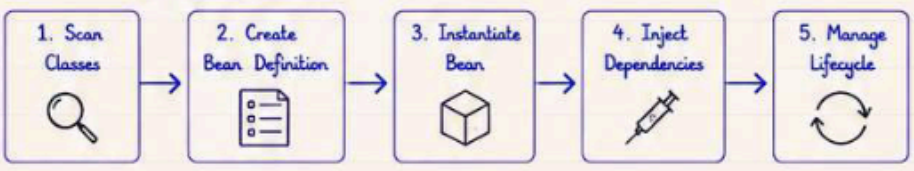
Key Point: Beans are managed by Spring container, not by the developer.

11. How does Spring create and manage beans?

Answer:

- Spring container scans the classes (using @ComponentScan).
- It identifies the classes annotated with stereotypes (@Component, @Service, @Repository, @Controller, etc.).
- It creates bean definitions.
- It instantiates the beans.
- It injects dependencies.
- It manages the lifecycle of beans.

Bean Creation Flow:



Top 50 Spring Boot Interview Questions

13. What is the Spring Bean Lifecycle?

Answer:

- The lifecycle of a bean starts from its creation by the Spring container to its destruction.
- Spring provides various callbacks to perform custom initialization and cleanup.
- Understanding the lifecycle helps in managing resources effectively.

Bean Lifecycle Steps:

1. Bean Instantiation
2. Populate Properties (Dependency Injection)
3. Bean NameAware / BeanFactoryAware (if applicable)
4. BeanPostProcessor – before initialization
5. @PostConstruct / InitializingBean.afterPropertiesSet()
6. Bean is ready to use
7. BeanPostProcessor – after initialization
8. @PreDestroy / DisposableBean.destroy()
9. Bean Destruction

14. What are @PostConstruct and @PreDestroy?

Answer:

- @PostConstruct is used to run a method after bean initialization.
- @PreDestroy is used to run a method before bean destruction.
- Both are part of javax.annotation package.

Example:

```
@Component
public class DemoBean {
    @PostConstruct
    public void init() {
        System.out.println("Bean Initialized");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Bean Destroyed");
    }
}
```

15. What is Bean Scope?

Answer:

- Scope defines the lifecycle and visibility of a bean.
- It determines how many instances of a bean can exist in the Spring container.

Common Scopes in Spring:

Scope	Description
singleton (default)	One instance per Spring IoC container.
prototype	New instance every time the bean is requested.
request	One instance per HTTP request.
session	One instance per HTTP session.
application	One instance per ServletContext.
websocket	One instance per WebSocket session.

16. Singleton vs Prototype scope?

Answer:

Singleton	Prototype
• Default scope in Spring. • Only one instance per container. • Stored in Spring container cache. • Same instance is returned for every request. • Used for stateless services, controllers, etc.	• New instance is created every time. • Not stored in container cache. • Different instance returned each time. • Used for stateful objects or heavy objects.

Example:

```
@Component
@Scope("singleton")   // default
public class SingletonBean { }
```

```
@Component
@Scope("prototype")
public class PrototypeBean { }
```

17. What are @Configuration and @Bean?

Answer:

- @Configuration is used to define configuration classes.
- @Bean is used to declare a bean that is managed by Spring container.
- These are part of Java-based configuration.

Example:

```
@Configuration
public class AppConfig {
    @Bean
    public MyService myService() {
        return new MyServiceImpl();
    }
}
```

18. @Component vs @Bean – when should you use each?

Answer:

- Use @Component (or its variants) for component scanning.
- Use @Bean for third-party classes or complex initialization.
- @Bean gives more control over bean creation.

@Component	@Bean
• Used with component scanning. • Classes are auto-detected. • Less control over creation. • Use when class is under our control.	• Defined in @Configuration class. • More control over creation. • Used for 3rd party classes or complex beans. • Use when class is not under our control.

Top 50 Spring Boot Interview Questions

19. What does @SpringBootApplication do internally?

Answer:

- @SpringBootApplication is a combination of three annotations:

@Configuration	→	Marks the class as a source of bean definitions.
@EnableAutoConfiguration	→	Enables Spring Boot auto-configuration mechanism.
@ComponentScan	→	Enables component scanning in the current package and sub-packages.

- It is the heart of any Spring Boot application.

Example:

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

20. What are @EnableAutoConfiguration, @CompenentScan and @Configuration?

Answer:

- @EnableAutoConfiguration
  - Enables Spring Boot to automatically configure beans based on classpath, dependencies and application properties.
  - It reduces manual configuration.
- @ComponentScan
  - Scans the current package and its sub-packages for components (like @Component, @Service, @Repository, @Controller).
  - Registers them as Spring beans.
- @Configuration
  - Indicates that the class declares one or more @Bean methods.
  - The class can be used by the Spring IoC container as a source of bean definitions.

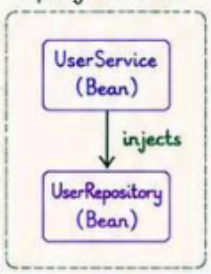
21. What is @Autowired and how does it work?

Answer:

- @Autowired is used to automatically inject dependencies into a bean.
- Spring looks for a matching bean in the application context and injects it.
- It can be used on constructors, fields, setter methods or methods.

Example (Field Injection):

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
}
```



22. What is @Qualifier and why is it needed?

Answer:

- @Qualifier is used along with @Autowired when multiple beans of the same type exist.
- It helps Spring to identify which bean to inject.

Example:

```
@Service
public class PaymentService {
    @Autowired
    @Qualifier("razorPay")
    private PaymentGateway paymentGateway;
}
```

Explanation:

If there are multiple PaymentGateway beans, @Qualifier tells Spring to inject the bean with name "razorPay".

23. What is @Primary?

Answer:

- @Primary is used to mark a bean as the default choice when multiple beans of the same type exist.
- It acts as a fallback when @Qualifier is not specified.

Example:

```
@Component
@Primary
public class StripePaymentGateway
        implements PaymentGateway {
    // implementation
}
```

→ When multiple PaymentGateway beans are present, Spring will inject StripePaymentGateway by default because of @Primary.

24. What is the difference between @Value and @ConfigurationProperties?

Answer:

Feature	@Value	@ConfigurationProperties
Purpose	Injects individual property values.	Binds a group of related properties to a class.
Usage	Used for a single value.	Used for multiple values (structured data).
Type Safety	Less type-safe.	Type-safe (POJO binding).
SpEL Support	Supports SpEL expressions.	Does not support SpEL.

Example	`@Value("${app.name}")`
	`private String appName;`
		`@ConfigurationProperties(prefix = "app")`
		`public class AppProperties {`
		`  private String name;`
		`  private String version;`
		`}`

Top 50 Spring Boot Interview Questions

25. How do you create a REST API using Spring Boot?

Answer:

- Create a @RestController class.
- Map HTTP methods using @GetMapping, @PostMapping, @PutMapping, @DeleteMapping.
- Use @RequestBody, @PathVariable, @RequestParam to handle data.
- Return response as JSON (by default).

Example:

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return userService.getUserById(id);
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return userService.createUser(user);
    }
}
```

26. @Controller vs @RestController?

Answer:

@Controller	@RestController
• Returns view name. • Used in traditional MVC applications. • Requires @ResponseBody to return data.	• Returns data directly (JSON/XML). • Combines @Controller and @ResponseBody. • Used in RESTful web services.

Example:

@Controller

```
@Controller
public class HomeController {
    @GetMapping("/home")
    public String home() {
        return "home";
    }
}

→ Returns view name "home"
```

@RestController

```
@RestController
public class UserController {
    @GetMapping("/users")
    public List<User> getUsers() {
        return userService.getAll();
    }
}

→ Returns JSON data
```

27. @RequestMapping vs @GetMapping, @PostMapping, etc.?

Answer:

- @RequestMapping is a generic annotation for all HTTP methods.
- @GetMapping, @PostMapping, @PutMapping, @DeleteMapping are specialized shortcuts.
- They make the code more readable.

Example:

Annotation	Purpose
@RequestMapping(method = RequestMethod.GET)	Handles GET request
@GetMapping	Shortcut for GET
@PostMapping	Shortcut for POST
@PutMapping	Shortcut for PUT
@DeleteMapping	Shortcut for DELETE

28. @PathVariable vs @RequestParam vs @RequestBody?

Answer:

Annotation	Purpose	Example
@PathVariable	Used to extract values from URL path.	/users/101 → @PathVariable Long id
@RequestParam	Used to extract query parameters.	/users?id=101 → @RequestParam Long id
@RequestBody	Used to receive data in request body (JSON/XML).	POST /users Body → { "name": "Aman" } → @RequestBody User user

Example:

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id,
                    @RequestParam String type) {
    // type will come from query param ?type=admin
    return userService.getUser(id, type);
}
```

29. How do you handle HTTP status codes in Spring Boot?

Answer:

- Use ResponseEntity<T> to return custom status codes.
- You can set status, headers, and body.
- Common status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error.

Example:

```
@GetMapping("/users/{id}")
public ResponseEntity<User> getUser(@PathVariable Long id) {
    User user = userService.getUserById(id);
    if (user == null) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
    }
    return ResponseEntity.status(HttpStatus.OK).body(user);
}
```

30. How do you handle exceptions globally using @ControllerAdvice?

Answer:

- Create a class annotated with @RestControllerAdvice or @ControllerAdvice.
- Use @ExceptionHandler to handle specific exceptions.
- It provides centralized exception handling.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                             .body(ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleAll(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                             .body("Something went wrong: " + ex.getMessage());
    }
}
```

Top 50 Spring Boot Interview Questions

31. What is Spring Data JPA?

Answer:

- Spring Data JPA is a part of Spring Data project.
- It provides easy implementation of JPA repositories.
- It reduces boilerplate code for CRUD operations and query creation.
- It works on top of JPA provider like Hibernate.

Example:

```
@Repository
public interface UserRepository
      extends JpaRepository<User, Long> {

    // Spring Data JPA will provide
    // implementation at runtime
}
```

32. What is the difference between JPA, Hibernate and Spring Data JPA?

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification (Standard)	ORM Framework (Implementation)	Repository Abstraction (Built on JPA)
Purpose	Defines rules and APIs.	Implements JPA specification.	Simplifies data access layer using JPA.
Focus	Standard for ORM.	Full ORM implementation.	Easy repository and less code.
Query	JPQL / Criteria API	HQL / Native SQL / Criteria	Method name queries, @Query annotation, etc.
Dependency	javax.persistence (jakarta.persistence)	org.hibernate (Core)	Depends on Spring Data + JPA

Example:

Spring Boot + Spring Data JPA + Hibernate (default provider)

33. What is JpaRepository?

Answer:

- JpaRepository is a part of Spring Data JPA.
- It provides CRUD operations, pagination, sorting and more.
- It extends PagingAndSortingRepository and CrudRepository.

Example:

```
@Repository
public interface ProductRepository
      extends JpaRepository<Product, Long> {
   // Built-in methods:
   // save(), findById(), findAll(), deleteById(), etc.
}
```

34. How does Spring Data JPA generate queries from method names?

Answer:

- Spring Data JPA parses the method name and creates query automatically based on naming convention.
- Keywords like findBy, And, Or, Between, Like, LessThan, etc. are used.

Example:

```
public interface UserRepository extends JpaRepository<User, Long> {
   List<User> findByName(String name);
   List<User> findByAgeGreaterThan(int age);
   List<User> findByNameAndActiveTrue(String name);
}
```

Spring converts above methods into appropriate JPQL queries.

35. What are @Entity, @Id, @GeneratedValue and @Column?

Answer:

- @Entity → Marks a class as JPA entity.
- @Id → Specifies the primary key.
- @GeneratedValue → Generates value for primary key (AUTO, IDENTITY, SEQUENCE, etc.)
- @Column → Maps a field to database column.

Example:

```
@Entity
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(name = "user_name", nullable = false, length = 50)
   private String name;

   private String email;
}
```

36. What are lazy loading and eager loading?

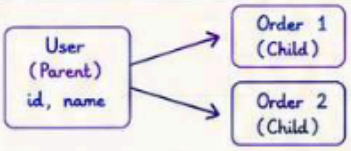
Answer:

- They are used to load related data (associations) between entities.
- Lazy Loading loads data only when it is needed.
- Eager Loading loads data immediately with the parent entity.

Lazy Loading	Eager Loading
• Data is loaded on demand. • Uses proxy objects. • Improves performance (loads only required data). • Annotation: fetch = FetchType.LAZY	• Data is loaded immediately. • May impact performance (loads more than needed). • Annotation: fetch = FetchType.EAGER

Example:

```
@OneToMany(fetch = FetchType.LAZY, mappedBy = "user")
private List<Order> orders;
```



Lazy → Orders loaded only when accessed.

Eager → Orders loaded with User.

Top 50 Spring Boot Interview Questions

37. What is @Transactional and how does it work?

Answer:

- @Transactional is used to manage database transactions.
- It ensures that either all operations in a method are successful (commit) or all are rolled back (rollback) in case of an error.
- It can be applied at class or method level.
- By default, it rolls back on RuntimeException and Error.

Example:

```
@Service
public class OrderService {
    @Autowired
    private OrderRepository orderRepository;

    @Transactional
    public void placeOrder(Order order) {
        orderRepository.save(order);
        // more DB operations
    }
}
```

If any RuntimeException occurs inside placeOrder() method, all DB operations will be rolled back.

38. What happens when an exception occurs inside a transaction?

Answer:

- If a RuntimeException (unchecked exception) occurs, the transaction is rolled back.
- If a checked exception occurs, the transaction will not be rolled back by default.
- You can customize rollback rules using rollbackFor and noRollbackFor.

Example:

```
@Transactional(
    rollbackFor = { Exception.class },
    noRollbackFor = { CustomCheckedException.class }
)
public void doSomething() {
    // code
}
```

Default Behavior
Rollback → RuntimeException, Error
No Rollback → Checked Exception

39. What are Spring Profiles?

Answer:

- Spring Profiles allow you to define different configurations for different environments (dev, test, prod, etc.).
- Only the beans/configuration specific to the active profile will be loaded.
- Helps in environment-specific behavior without changing code.

Example:

```
# application-dev.yml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/devdb
---
# application-prod.yml
spring:
  datasource:
    url: jdbc:mysql://prod-server:3306/proddb
```

Activate profile using:

1. application.properties
   spring.profiles.active=dev
2. Command Line
   --spring.profiles.active=prod
3. Environment Variable
   SPRING_PROFILES_ACTIVE=dev

40. How do application.properties and application.yml work?

Answer:

- These files store configuration properties for Spring Boot application.
- Spring Boot loads these properties at startup.
- application.yml is YAML format (hierarchical) and supports profiles easily.
- application.properties is key=value format.

Example:

application.properties

```
server.port=8080
spring.datasource.url=
jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=root
```

application.yml

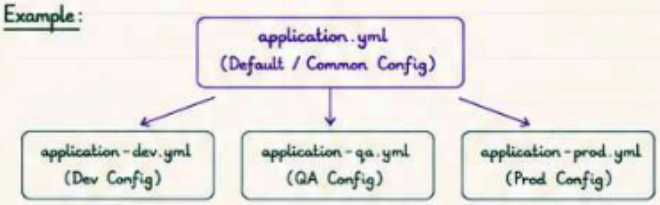
```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: root
```

41. How do you manage different configurations for dev, QA and production?

Answer:

- Use Spring Profiles (dev, qa, prod).
- Keep separate property files: application-dev.yml, application-qa.yml, application-prod.yml.
- Activate the appropriate profile based on environment.
- You can also externalize configuration using environment variables or command line arguments.

Example:



Activate using: spring.profiles.active=dev | qa | prod

42. How do you manage secrets and sensitive configuration?

Answer:

- Never hardcode sensitive data like passwords, API keys in code or property files.
- Use environment variables, external configuration servers or secret management tools.
- Popular options: Spring Cloud Config, AWS Secrets Manager, HashiCorp Vault, Azure Key Vault.

Example:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
```

→ Values provided via Environment Variables or Secret Manager.

Ways to provide secrets:

1. Environment Variables
2. Command Line Arguments
3. External Config Server (Spring Cloud Config)
4. Secrets Manager (AWS / Azure / Vault)

Better Security & Best Practice

Top 50 Spring Boot Interview Questions

43. What is Spring Boot Actuator?

Answer:

- Spring Boot Actuator provides production-ready features to help you monitor and manage your application.
- It exposes various endpoints to check health, metrics, info, beans, mappings, etc.
- It helps in monitoring without any invasive code.

Common Actuator Endpoints:

Endpoint	Description
/actuator/health	Shows application health information
/actuator/info	Displays application info
/actuator/metrics	Shows application metrics
/actuator/beans	Lists all Spring beans
/actuator/mappings	Displays all request mappings

Example:

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

44. What is the difference between @RestController and @Controller?

Answer:

@Controller	@RestController
Used to return views.	Used to return data (JSON/XML).
It returns the name of view (HTML).	It returns response body directly.
Uses ModelAndView or Model	Uses @ResponseBody internally.
Used in traditional MVC applications.	Used in RESTful web services.
Needs @ResponseBody for returning data.	No need to use @ResponseBody.

Example:

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String home() {
        return "index"; // view name
    }
}
```

```
@RestController
public class UserController {
    @GetMapping("/users")
    public List<String> getUsers() {
        return List.of("Aman", "John");
    }
}
```

45. What is Spring Boot Starter?

Answer:

- Starter is a set of convenient dependency descriptors for a particular feature.
- It simplifies dependency management.
- It brings all the required dependencies for a specific feature in a single dependency.

Example:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

This starter brings:
- spring-web
- spring-webmvc
- jackson-databind
- validation
- and more...

46. What is the purpose of @SpringBootApplication?

Answer:

- It is a combination of three annotations:

```
@Configuration      → Indicates configuration class
@EnableAutoConfiguration → Enables auto-configuration
@ComponentScan      → Scans components in base package
```

- It is used on the main class to enable Spring Boot features.
- It is equivalent to using these three annotations together.

Example:

```
@SpringBootApplication
public class MyApplicatler {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

47. What is the difference between @Autowired and @Resource?

Answer:

	@Autowired	@Resource
Package	org.springframework.beans.factory.annotation	javax.annotation
Default	By Type	By Name
Injection Type	Constructor, Setter, Field	Setter, Field
Required	required=true (default)	required=true (default)
Used For	Spring Beans	Any Java Object
Features	Supports collections, optional	JSR-250 standard annotation

Example:

```
@Autowired
private UserService userService;
```

```
@Resource
@Resource(name = "userService")
private UserService userService;
```

Use @Autowired in Spring apps. Use @Resource for JSR compliance.

48. What is a Filter, Interceptor and Listener in Spring Boot?

Component	Purpose
Filter	Works at servlet level. Used for request pre-processing and post-processing. (e.g., logging, auth)
Interceptor	Works at Spring MVC level. Used to intercept requests before controller execution. (e.g., logging, authentication)
Listener	Listens to application events. Used to perform actions on events. (e.g., ContextInitializedEvent)

49. What is DevTools in Spring Boot?

Answer:

- DevTools is a set of tools to enhance developer productivity.
- It provides features like automatic restart, live reload, property defaults, etc.
- It should not be used in production.

50. What are the ways to externalize configuration?

Answer:

- application.properties or application.yml
- Environment Variables
- Command Line Arguments → java -jar app.jar --server.port=8081
- System Properties → -Dserver.port=8081
- Profile specific properties → application-dev.yml